

PER2025-057 - Développement

Systèmes collaboratifs pour le contrôle d'atterrissage d'un nano-drone sur plateforme mobile

Étudiants: Komi Jean Paul ASSIMPAH (SI5 IoT-CPS),
Alban FALCOZ (SI5 IA-ID),
Evan GALLI (SI5 IoT-CPS),
Alexandre GRIPARI (SI5 IA-ID)

Encadrant: Gérald ROCHER

I. Résumé exécutif

Ce projet vise à développer un système de contrôle collaboratif permettant à un nano-drone (Crazyflie 2.1+) d'atterrir de manière autonome sur une plateforme mobile (Waveshare AlphaBot2). Contrairement aux approches unilatérales classiques, notre solution coordonne les trajectoires des deux robots pour optimiser le temps et la précision de l'atterrissage.

La stratégie de guidage sera développée par apprentissage par renforcement profond (Deep Reinforcement Learning) au sein de l'environnement NVIDIA IsaacSim/IsaacLab. L'architecture système reposera sur ROS 2 Humble pour assurer la communication temps-réel et la synchronisation des données d'odométrie entre les agents.

Livrables attendus :

Environnement de simulation complet : Configuration d'IsaacSim/IsaacLab intégrant le Crazyflie 2.1+ et la plateforme mobile omnidirectionnelle. La plateforme sera équipée virtuellement d'une caméra horizontale et de capteurs d'odométrie réalistes.

Modèle de contrôle par Deep Reinforcement Learning (Deep RL) : Obtention d'une politique de contrôle coordonnée capable d'assurer un atterrissage dynamique avec un taux de réussite supérieur à 80% en simulation. Le modèle utilisera les observations d'odométrie relative et les données inertielles des deux robots.

Validation du transfert Sim2Sim : Évaluation de la robustesse du modèle en le déployant dans l'environnement Gazebo. Cette étape permettra d'identifier les premières limitations avant le passage au matériel réel.

Analyse du reality gap (Sim2Real) : Déploiement expérimental sur le matériel physique et documentation détaillée des écarts observés (latences, imprécisions capteurs, effets aérodynamiques non modélisés). L'objectif n'est pas la performance absolue mais la

caractérisation rigoureuse du reality gap et l'identification de pistes d'amélioration (domain randomization, fine-tuning adaptatif).

Ce projet contribue à la recherche sur les systèmes multi-robots hétérogènes en explorant méthodiquement les défis du transfert de modèles de contrôle appris en simulation vers des plateformes matérielles réelles (Sim2Real), un problème ouvert majeur en robotique par apprentissage.

II. Description du projet

Contexte technologique

Nano-drone Crazyflie 2.1+ : C'est un nano-drone modulaire et open-source de 27g, utilisé en recherche pour l'expérimentation en robotique distribuée et en contrôle embarqué. Le drone sera équipé des decks suivants : Flow Deck v2 (estimation de mouvement par flux optique et télémètre) et Multi-ranger Deck (détection d'obstacles multidirectionnelle). Communication radio via Crazyradio PA. (====Partie tirée de nos discussions à confirmer====)

Plateforme mobile AlphaBot2 : Robot mobile omnidirectionnel équipé de quatre roues mecanum permettant des déplacements dans toutes les directions sans rotation. La plateforme intégrera une caméra horizontale pour la détection du drone et un système d'odométrie roues-encodeurs pour estimer sa position. Contrôle via Raspberry Pi avec ROS 2 Humble. (====Caméra suffisant à confirmer====)

ROS 2 Humble : Middleware robotique permettant la communication distribuée entre les composants logiciels. Architecture basée sur des topics pour la publication/souscription des données capteurs :

- /drone/odom : Position (X, Y, Z) et orientation (roll, pitch, yaw) du drone
- /platform/odom : Position (X, Y) et vitesse de la plateforme
- /relative_pose : Position relative calculée entre les deux agents
- /cmd_vel : Commandes de vitesse pour chaque robot

NVIDIA IsaacSim/IsaacLab : Environnement de simulation physique GPU-accelerated basé sur Omniverse. IsaacLab fournit un framework spécialisé pour l'apprentissage par renforcement profond avec support natif d'algorithmes classiques (PPO, SAC, TD3). L'accélération GPU permet d'exécuter des milliers d'environnements en parallèle, réduisant drastiquement le temps d'entraînement comparé aux approches séquentielles traditionnelles.

Gazebo : Simulateur robotique open-source utilisé pour la validation Sim2Sim. Permet de tester la robustesse du modèle dans un environnement physique différent avant le déploiement réel.

Motivations scientifiques et techniques

Collaboration interactive vs. atterrissage unilatéral : La majorité des travaux existants sur l'atterrissage de drones sur plateformes mobiles adoptent une approche unilatérale où seul le drone ajuste sa trajectoire tandis que la plateforme suit un chemin prédéfini. Notre projet explore une coordination bidirectionnelle où les deux agents négocient leurs trajectoires pour optimiser le temps et la précision d'atterrissage. Cette approche est cruciale pour des applications réelles comme le rechargement mobile autonome ou la logistique multi-domaines.

Contribution à la recherche sur le Sim2Real : Le transfert de politiques apprises en simulation vers le monde réel (Sim2Real) reste un défi majeur en robotique par apprentissage. Notre projet contribue à cette problématique en :

- Quantifiant méthodiquement le reality gap sur une tâche de coordination dynamique
- Comparant deux environnements de simulation (IsaacSim vs Gazebo) pour évaluer la variabilité Sim2Sim
- Identifiant les sources principales d'écart (latences, imprécisions capteurs, effets aérodynamiques)

Optimisation temporelle par Time Optimal Control : Le projet vise à générer des trajectoires minimales en temps tout en respectant les contraintes dynamiques des deux robots. Cette approche garantit un processus d'atterrissage efficient, critère essentiel pour des applications énergétiquement contraintes.

Optimisation pour l'Edge Computing embarqué : (====à confirmer, il me semble que ce soit pour ça qu'on attend la carte NVIDIA===) L'exécution d'inférence de modèles Deep RL en temps-réel sur des plateformes embarquées à ressources contraintes est un défi pour le déploiement réel de systèmes autonomes. Dans le contexte de notre projet, la plateforme mobile AlphaBot2 pourrait être équipée d'une carte de calcul embarquée (type NVIDIA Jetson Nano) pour exécuter localement le modèle de décision.

Objectifs du projet

Objectif principal

Développer, sous ROS 2 Humble, un système de contrôle collaboratif permettant l'atterrissage autonome du Crazyflie 2.1+ sur la plateforme mobile AlphaBot2 en mouvement. La stratégie de guidage sera apprise par renforcement profond (Deep RL) dans l'environnement NVIDIA IsaacSim couplé à IsaacLab.

Objectifs secondaires

Conformément au sujet, le projet comportera les étapes suivantes :

Architecture ROS 2 et plateformes collaboratives : Développement des nœuds de contrôle et de communication permettant la coordination entre le Crazyflie et l'AlphaBot2, incluant l'exploitation de la caméra horizontale et du Flow v2 Deck.

Environnements de simulation et apprentissage : Configuration d'IsaacSim/IsaacLab pour l'entraînement d'un modèle de décision par Deep RL, avec intégration des capteurs nécessaires.

Validation Sim2Sim : Déploiement et test du modèle dans Gazebo pour analyser les problématiques de transfert entre simulateurs.

Validation Sim2Real et analyse du reality gap : Déploiement sur matériel réel (Crazyflie 2.1 + AlphaBot2) avec caractérisation quantitative des écarts simulation/réalité.

L'évaluation portera prioritairement sur la démarche méthodologique et l'analyse du reality gap plutôt que sur la performance absolue en conditions réelles.

Risques identifiés (et contre-mesures)

1. Complexité du Sim2Real : Le transfert des modèles appris en simulation vers le matériel réel est un défi majeur compte tenu de l'ensemble des événements imprévus dans la réalité qui ne peuvent pas être appris en simulation.

Contre-mesure : L'accent sera mis sur l'analyse et la compréhension des déviations (pourquoi ça ne marche pas), et l'utilisation de techniques comme la randomisation du domaine pour augmenter la robustesse

2. Difficulté des rewards : La définition des fonctions de rewards est complexe, surtout pour des comportements précis comme l'atterrissage dynamique.

Contre-mesure : S'appuyer sur le modèle de rewards du quad copter existant dans Isaac Lab et réfléchir en amont aux observations supplémentaires nécessaires (odométrie relative, position)

3. Synchronisation et Données : Assurer la communication et la synchronisation des données d'odométrie entre le drone et la plateforme mobile via ROS.

Contre-mesure : Définir clairement les topics ROS nécessaires (position X, Y, Z, inclinaison) et faire l'hypothèse d'une condition initiale où les deux robots sont localisés l'un par rapport à l'autre.

4. Modélisation et intégration de la plateforme mobile : L'instanciation du robot mobile AlphaBot2 et son contrôle indépendant dans IsaacSim peut être un défi.

Contre-mesure : Commencer par instancier un modèle existant simple (comme le Turtle Bot 3) dans Gazebo/IsaacSim pour vérifier la faisabilité du contrôle multi-robot. S'assurer que les observations sont correctement associées à chaque robot.

Scenarios

[Décrivez 2 à 3 scénarios d'utilisation de votre projet. Ces scénarios doivent être montrés du point de vue des utilisateurs du système que vous construirez. Pour chaque scénario, vous soulignerez les critères d'acceptation, qui servent à prouver que le système permet l'exécution de ces scénarios. Maximum deux pages.]

III. Mise en œuvre

(quelques paragraphes, utilisez des bulles)

Liste d'activités déjà réalisées avant les semaines à plein temps

- Configuration ROS/Crazyflie : Installation de l'environnement ROS, permettant de faire fonctionner les tutoriels de base du Crazyflie et d'accéder aux topics des données capteurs (odométrie, distance).
- Configuration IsaacSim/IsaacLab : Installation du framework et vérification de la compatibilité matérielle.
- P.O.C. DeepRL : Entraînement réussi du modèle DeepRL par défaut dans IsaacLab, impliquant l'atterrissage d'un quadcoptère sur un cube statique.
- Analyse du Modèle : Identification des 12 observations (vélocité linéaire/angular, distance relative au but) et des 4 actions (puissance, mouvements X Y Z) utilisées par le quadcoptère par défaut dans IsaacLab.

Listes d'activités prévues pour chaque semaine à plein temps

Organisation du travail (répartition de l'équipe) Le travail est organisé entre les 4 membres du groupe selon leurs mineurs :

- Mineur IA-ID (Environnement Simulation & DeepRL) : Prend en charge l'installation, la modélisation de la plateforme mobile dans IsaacSim/Lab, la modification des tasks et des rewards, et l'entraînement du modèle DeepRL.
- Mineur IoT-CPS (Systèmes Robotiques & ROS) : Prend en charge l'architecture ROS 2, le développement du nœud de calcul de position relative, la vérification Sim2Sim (Gazebo), et le setup matériel de l'AlphaBot2